Hunter College

Department of Computer Science

CSCI 39548: Practical Web Development

Summer 2026

Exit Assessment — Snack Tracker

Instructor: *Sourya Saha (sourya.saha24@login.cuny.edu)*

Date	Last class session (Zoom, live)
Duration	**2 hours**, hard stop
Weight	Pass/fail gate — you must score $\geq 60\%$ to pass the course
Format	In-class, solo, **screen-share recorded** on Zoom for the full session
Tools allowed	Browser, VS Code (or any editor), Live Server / `python -m http.server`
Tools NOT allowed	**No AI of any kind** (Copilot, ChatGPT, Claude, Cursor, Gemini, etc.). No code from prior assignments. No internet. No Stack Overflow / Google / Discord / messaging anyone. No frameworks, no libraries.

1 What You're Building

A single-page **Snack Tracker** — pure HTML + CSS + vanilla JavaScript. No backend, no framework, no library. You load a static `data.json`, render a list, support a live search, accept a form submission with validation, and let the user mark snacks as eaten.

> This assignment exists to certify that **you personally** can build a small interactive web page from scratch. It is the floor of the course.

2 What You're Given

A **single file** on the course GitHub page at `exit-assignment/data.json`:

- `data.json` — top-level resource `snacks`, 15 entries, shape:

```
{ "id": 1, "name": "Granola Bar", "category": "Bar", "
   calories": 190 }
```

That is the *entire* starter kit.

3 What You Must Create

- `index.html`
- `style.css`
- `app.js`

All three from scratch in the same folder as `data.json`.

4 Required IDs, Classes, and Names

These are **mandatory**. The grader runs against these exact selectors. Each deviation = −2 **pts**.

Selector	What it is
`#list`	The `<ul>` that holds rendered snack `<li>`s
`#search`	The `<input>` used for live search
`#form`	The `<form>` used to add a new snack
`input name="name"`	Text input inside the form
`input name="category"`	Text input inside the form
`input name="calories"`	Numeric input inside the form
`#error`	A `<p>` (or `<div>`) where validation errors are shown
`.eaten`	CSS class applied to an `<li>` when clicked

5 Tasks (100 pts total)

Each task is graded independently.

5.1 Task 0 — Markup and Styles (10 pts)

Write `index.html` and `style.css`.

- Semantic structure: a `<header>`, a `<main>`, a `<form>` with `<label>`s tied to inputs, the search input, the `#list` `<ul>`, and the `#error` element.
- `<script src="app.js" defer></script>` linked in `<head>`.
- `.eaten` must visually distinguish eaten snacks (e.g., `text-decoration: line-through; opacity: 0.5;`).
- The page must look intentional — not unstyled. Spacing, font, at least one color, a readable layout.

5.2 Task 1 — Load and Render (25 pts)

On page load:

1. `fetch("./data.json")` and parse the response.
2. Store the `snacks` array in a module-level variable (you will mutate it in Task 3).
3. Render each snack as an `<li>` inside `#list`, with text content:

```
Granola Bar --- Bar --- 190 cal
```

The exact separator can be `---`, `|`, or anything readable — the order `name --- category --- calories cal` is required.

If the fetch fails, write an error message into `#error`.

5.3 Task 2 — Live Search (25 pts)

The `#search` input filters the list **as the user types** (no submit, no button).

- Match on `name`, **case-insensitive**, **substring** (not prefix-only).
- An empty search field shows all snacks.
- Re-render by clearing `#list` and rebuilding it from the filtered array.

5.4 Task 3 — Add a Snack (25 pts)

When `#form` is submitted:

1. Prevent the default page reload.
2. Read all three fields.
3. **Reject** the submission and show a message in `#error` if:
   - any field is empty (after trimming), OR
   - `calories` is not a positive number.
4. On valid submit:
   - Push a new snack object into the in-memory array. Assign a unique `id` (e.g., `Date.now()` or `max(existing ids) + 1`).
   - Clear the form fields.
   - Clear `#error`.
   - Re-render the list (the new snack must appear, and it must respect the current search filter).

The new snack only lives in memory — a page reload is allowed to lose it.

5.5 Task 4 — Toggle Eaten (15 pts)

Clicking any `<li>` toggles the `.eaten` class on that `<li>`.

- Toggling is per-item: clicking one snack does not affect the others.
- The toggled state must survive a subsequent re-render **only if** you mutate the in-memory data (e.g., add an `eaten: true` field on the object). If you only toggle the DOM class, the state is allowed to reset on re-render — both approaches earn full credit.

6 Constraints

- **No frameworks, no libraries.** Vanilla HTML/CSS/JS only. No React, no jQuery, no Tailwind, no Bootstrap, no Lodash, no axios.
- **No AI.** Copilot must be disabled. No ChatGPT/Claude/Cursor tabs.
- **No internet.** No browsing of any kind during the assessment.
- **No code from prior assignments.** Write everything in-session.
- `async/await` or `.then()` are both fine.
- Re-render by clearing `#list` and rebuilding — no diffing required.
- Your code should run when you open the folder in Live Server (or `python -m http.server`) and visit `index.html`. **`fetch` on `file://` will not work** — you must serve the folder.

7 Submission

At the end of the 2-hour window:

1. Stop typing.
2. Push your three files (`index.html`, `style.css`, `app.js`) to your GitHub repository under an `exit-assignment/` folder.
3. On **Brightspace**, paste the link to your `exit-assignment/` folder on GitHub.

Note: Late submissions are not accepted. The window closes when class ends.

8 Grading Rubric

Bucket	**Pts**	**Notes**
Task 0 — markup and styles	10	Semantic HTML; styled and readable; `.eaten` visually distinguishable
Task 1 — load and render	25	Fetch works; all 15 snacks render; correct text format; renders into `#list`
Task 2 — live search	25	Case-insensitive substring match; live on input; empty search shows all
Task 3 — add a snack	25	`preventDefault`; validates empty + non-positive calories; shows error; appends; clears; re-renders
Task 4 — toggle eaten	15	Click `<li>` toggles `.eaten`; per-item; no other side effects
Deductions	—	-2 per deviation from required IDs/classes/names
Total	**100**	**Pass: $\geq$ 60**

9 Pass/Fail Rule

If your final score is **less than 60**, you fail the course regardless of your weighted grade on the four assignments and the final project. There is no make-up exit assessment.

If your final score is $\geq$ **60**, the exit does not add to your grade — the four assignments and final project compute it. The exit only certifies eligibility.

End of ***Exit Assessment***